

PROJECT 1: UPDATED DESIGN

Multi-Agent Architecture · Firestore · Qdrant · Gemini Stack

PART 1: VISITOR EXPERIENCE (Unchanged)

User lands → Popup → Enters email + role → Backend processes → Returns **complete website config** → Renders personalized portfolio

PART 2: BACKEND ARCHITECTURE (Multi-Agent System)

Overall Flow

VISITOR SUBMITS EMAIL

|

▼

POST /api/personalize

|

▼

Check Firestore Cache

└ HIT (within 24h) → Return instantly

└ MISS → Start LangGraph Supervisor

|

▼

LANGGRAPH SUPERVISOR AGENT
(Orchestrates entire personalization flow)

|



RESEARCH

COMPANY

PORTFOLIO

SUPERVISOR

RESEARCH

RAG

AGENT

AGENT

AGENT

|

|

|

|

▼

VALIDATION AGENT

(Check: enough info?)

|

▼

PERSONALIZATION AGENT

(Generate website config)

|

▼

FIRESTORE

(Store 24h)

|

▼

FRONTEND

(Render everything)

****AGENT 1: Research Supervisor Agent****

Responsibility: Orchestrate, decide what info needed, coordinate other agents

```
class ResearchSupervisorAgent(Agent):  
    """  
    Root LangGraph node.  
    Decides:  
    - What information do I need?  
    - What do I already know?  
    - What is missing?  
    - Which agent should run next?
```

```

- Is confidence sufficient?
"""
async def run(self, input_data):
    email = input_data["email"]
    domain = email.split("@")[1] # Extract company from email
    # GEMINI CALL 1: Research Planning
    plan = await gemini.generateContent(f"""
    Email: {email}
    Domain: {domain}
    User role: {input_data.get('role')}
    What information do I need to personalize?
    - Visitor profile (title, company, seniority)?
    - Company context (what they build, tech, hiring)?
    - Portfolio evidence (which projects are relevant)?
    Return JSON:
    {{
    "need_visitor_profile": true/false,
    "need_company_research": true/false,
    "need_portfolio_chunks": true/false,
    "search_queries_for_portfolio": [...]
    }}
    """)
    # Launch parallel agents based on plan
    tasks = []
    if plan.need_company_research:
        tasks.append(
            self.company_agent.run({
                "domain": domain,
                "company": input_data.get("company")
            })
        )
    if plan.need_portfolio_chunks:
        tasks.append(
            self.portfolio_agent.run({
                "search_queries": plan.search_queries_for_portfolio
            })
        )
    # Gather visitor profile from email (basic info)
    visitor_profile = {
        "email": email,
        "domain": domain,
        "role": input_data.get("role"),
        "company": input_data.get("company")
    }
    # Wait for all research to complete
    research_results = await asyncio.gather(*tasks)
    return {
        "visitor_profile": visitor_profile,
        "company_research": research_results[0] if research_results else None,
        "portfolio_chunks": research_results[1] if len(research_results) > 1 else []
    }

```

****AGENT 2: Company Research Agent****

Responsibility: Research company WITHOUT LinkedIn scraping

Tools Used:

- Playwright (headless browser for scraping)
- Google Search API (find engineering blog)
- Company website scraping

```
class CompanyResearchAgent(Agent):
    """
    Scrapes company website, careers page, engineering blog.
    ✖ NO LinkedIn scraping (too fragile, breaks often)
    ✔ YES:
    - Company website
    - Careers/jobs page
    - Engineering blog
    - Crunchbase/news
    """

    async def run(self, input_data):
        domain = input_data["domain"]
        # Parallel scraping
        tasks = [
            self._scrape_website(domain),
            self._scrape_careers(domain),
            self._scrape_blog(domain),
            self._search_news(domain)
        ]
        results = await asyncio.gather(*tasks, return_exceptions=True)
        # Filter out errors
        results = [r for r in results if not isinstance(r, Exception)]
        # GEMINI CALL 2: Segmentation (convert scraped data to structured JSON)
        structured = await gemini.generateContent(f"""
        Convert scraped company data into structured JSON:
        Website content: {results[0]}
        Careers page: {results[1]}
        Engineering blog: {results[2]}
        News: {results[3]}
        Return JSON:
        {{
        "company": "...",
        "industry": "...",
        "what_they_build": "...",
        "tech_stack": ["Python", "Kubernetes", ...],
        "hiring_focus": ["RAG engineers", "ML ops", ...],
        "recent_events": [
        {{ "event": "Series D funding", "date": "June 2024" }},
        ...
        ],
        "company_size": 1500,
        "growth_stage": "scale-up",
        "engineering_culture": "fast-moving, AI-first"
        }}
        """)
        return structured

    async def _scrape_website(self, domain):
        """Scrape /about, /products, main content"""
        try:
            async with async_playwright() as p:
                browser = await p.chromium.launch()
```

```

page = await browser.new_page()
await page.goto(f"https://{domain}", timeout=10000)
content = await page.content()
await browser.close()
return content
except Exception as e:
return f"Failed to scrape website: {e}"
async def _scrape_careers(self, domain):
    """Scrape /careers or /jobs to see what roles they're hiring"""
    career_urls = [
        f"https://{domain}/careers",
        f"https://{domain}/jobs",
        f"https://jobs.{domain}",
        f"https://careers.{domain}"
    ]
    for url in career_urls:
        try:
            async with async_playwright() as p:
                browser = await p.chromium.launch()
                page = await browser.new_page()
                await page.goto(url, timeout=10000)
                content = await page.content()
                await browser.close()
                return content
        except:
            continue
    return "No careers page found"
async def _scrape_blog(self, domain):
    """Scrape engineering blog to see tech stack"""
    blog_urls = [
        f"https://blog.{domain}",
        f"https://engineering.{domain}",
        f"https://{domain}/blog",
        f"https://{domain}/engineering"
    ]
    for url in blog_urls:
        try:
            async with async_playwright() as p:
                browser = await p.chromium.launch()
                page = await browser.new_page()
                await page.goto(url, timeout=10000)
                content = await page.content()
                await browser.close()
                return content
        except:
            continue
    return "No blog found"
async def _search_news(self, domain):
    """Search for recent news about company"""
    # Use Google News API or Bing News
    # Return funding announcements, product launches, hiring announcements
    return {}

```

****AGENT 3: Portfolio RAG Agent****

Responsibility: Retrieve relevant portfolio evidence (one-time setup + per-request retrieval)

```

class PortfolioRAGAgent(Agent):
    """
    Portfolio documents chunked, embedded, stored in Qdrant.
    Retrieves best evidence relevant to this specific visitor.
    PURPOSE: Retrieval (for personalization)
    NOT: Answering questions (that's the chatbot)
    """

    def __init__(self):
        self.qdrant_url = "qdrant:6333"
        self.embeddings = HuggingFaceEmbeddings(
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )
        self._init_portfolio_vectors() # One-time setup
        async def _init_portfolio_vectors(self):
            """
            ONE-TIME: Chunk all portfolio docs, embed, store in Qdrant
            Portfolio documents:
            - Case study: Smart Portfolio RAG (project 1)
            - Case study: Agentic RAG (project 2)
            - Resume (detailed)
            - Blog posts (architecture, best practices)
            - Architecture docs
            - GitHub READMEs
            """

            portfolio_docs = [
                {
                    "id": "project-1-casestudy",
                    "title": "Smart Portfolio with RAG",
                    "content": """
                    Smart Portfolio with RAG
                    Problem: Portfolio needs to be dynamic, personalized per visitor.
                    Solution: Built AI-powered personalization engine.
                    Architecture:
                    - Frontend: Next.js + React
                    - Backend: FastAPI
                    - LLM: Gemini
                    - Vectoring: Qdrant
                    - Storage: Firestore
                    Results:
                    - Shipped in 6 days
                    - 82% retrieval precision
                    - 200ms latency p99
                    - 99.9% uptime
                    """,
                    "type": "project",
                    "keywords": ["RAG", "personalization", "full-stack", "LLM"]
                },
                {
                    "id": "project-2-casestudy",
                    "title": "Agentic RAG System with RAGAS",
                    "content": """
                    Agentic RAG System with RAGAS Evaluation
                    Built a multi-agent system using LangGraph.
                    Evaluates RAG quality with RAGAS framework.
                    Features:
                    - Supervisor agent orchestration
                    - Multi-tool agent calls
                    - Evaluation layer (RAGAS)
                    """
                }
            ]

            # Store portfolio documents in Qdrant
            self._store_portfolio_docs(portfolio_docs)

            # Initialize portfolio vectors
            self._init_portfolio_vectors()

            # Return portfolio documents
            return portfolio_docs

        self._init_portfolio_vectors()

    def _store_portfolio_docs(self, docs):
        """Store portfolio documents in Qdrant"""
        # Implementation details
        pass

    def _init_portfolio_vectors(self):
        """Initialize portfolio vectors"""
        # Implementation details
        pass

    def _get_portfolio_docs(self):
        """Retrieve portfolio documents from Qdrant"""
        # Implementation details
        pass

```

```

- Production deployment
Metrics:
- RAGAS average: 0.82
- Context retrieval precision: 85%
- Faithfulness: 0.89
"""
"type": "project",
"keywords": ["agent", "RAG", "RAGAS", "evaluation", "LangGraph"]
},
{
  "id": "resume-full",
  "title": "Resume - Detailed",
  "content": ""
Aditya - Full-Stack AI Engineer
Experience:
- 2 years at Valiance Analytics
- Built RAG systems, deployed to production
- Full-stack: React, Node.js, Python, GCP
- Deep expertise: LangChain, FastAPI, Docker
Skills:
- GenAI: LangChain, LangGraph, RAG, agents
- Backend: FastAPI, Node.js, Docker
- Frontend: React, TypeScript, Tailwind
- Cloud: GCP (Cloud SQL, Cloud Storage, OCR)
- DevOps: Docker, monitoring, logging
"""
"type": "resume",
"keywords": ["experience", "skills", "full-stack", "GenAI"]
},
{
  "id": "blog-rag-production",
  "title": "Building Production RAG Systems",
  "content": ""
Key considerations for production RAG:
1. Retrieval Quality
- BM25 + semantic search hybrid
- Reranking with cross-encoders
- RAGAS evaluation
2. Latency
- Caching at retrieval layer
- Batch embeddings
- Vector DB indexing
3. Cost
- Smaller models for embedding
- Quantization
- Efficient token usage
"""
"type": "blog",
"keywords": ["RAG", "production", "retrieval", "latency", "cost"]
},
{
  "id": "arch-personalization",
  "title": "System Architecture: Personalization Engine",
  "content": ""
Multi-tier personalization architecture:
Tier 1: Company Research Agent
- Scrapes company website, careers, blog
- Extracts tech stack, hiring focus
Tier 2: Portfolio Retrieval
- Vector search for relevant projects

```

```

- Ranking by relevance score
Tier 3: LLM Synthesis
- Combines all context
- Generates personalized website config
Tier 4: Caching
- Firestore for 24h personalization caching
- Reduces LLM calls 70%
"""
"type": "architecture",
"keywords": ["architecture", "personalization", "system-design"]
}
]
# Chunk documents
chunks = []
for doc in portfolio_docs:
doc_chunks = self._chunk_document(doc, chunk_size=500, overlap=50)
chunks.extend(doc_chunks)
print(f"Created {len(chunks)} chunks from {len(portfolio_docs)} documents")
# Embed all chunks
embedded_chunks = []
for chunk in chunks:
vector = self.embeddings.embed_query(chunk["text"])
embedded_chunks.append({
"id": chunk["id"],
"vector": vector,
"payload": {
"text": chunk["text"],
"doc_id": chunk["doc_id"],
"type": chunk["type"],
"keywords": chunk["keywords"]
}
})
# Store in Qdrant
from qdrant_client.models import PointStruct
points = [
PointStruct(
id=i,
vector=chunk["vector"],
payload=chunk["payload"]
)
for i, chunk in enumerate(embedded_chunks)
]
from qdrant_client import QdrantClient
client = QdrantClient(self.qdrant_url)
# Create collection
try:
client.delete_collection("portfolio")
except:
pass
from qdrant_client.models import VectorParams, Distance
client.create_collection(
collection_name="portfolio",
vectors_config=VectorParams(
size=len(embedded_chunks[0]["vector"]),
distance=Distance.COSINE
)
)
# Upsert points
client.upsert(
collection_name="portfolio",

```



```

points=points
)
print("Portfolio vectors stored in Qdrant")
async def run(self, input_data):
    """
    Given search queries, retrieve most relevant portfolio chunks.
    """
    search_queries = input_data.get("search_queries", [])
    if not search_queries:
        return []
    from qdrant_client import QdrantClient
    client = QdrantClient(self.qdrant_url)
    all_results = []
    for query in search_queries:
        # Embed the query
        query_vector = self.embeddings.embed_query(query)
        # Search Qdrant
        results = client.search(
            collection_name="portfolio",
            query_vector=query_vector,
            limit=3, # Top 3 chunks per query
            score_threshold=0.5
        )
        # Add results
        for result in results:
            all_results.append({
                "id": result.id,
                "text": result.payload["text"],
                "doc_type": result.payload["type"],
                "relevance_score": result.score
            })
        # Remove duplicates, keep top 5 most relevant
        unique_results = {}
        for result in all_results:
            if result["id"] not in unique_results:
                unique_results[result["id"]] = result
        # Sort by relevance
        sorted_results = sorted(
            unique_results.values(),
            key=lambda x: x["relevance_score"],
            reverse=True
       )[:5]
        return sorted_results

```

****AGENT 4: Validation Agent****

Responsibility: Check if we have sufficient information before personalization

```

class ValidationAgent(Agent):
    """
    Validates: Do we have enough info to personalize well?
    Checks:
    - Do we have visitor profile?
    - Do we have company research?
    - Do we have portfolio chunks?
    - Is confidence >= threshold?
    """
    async def run(self, input_data):
        visitor_profile = input_data.get("visitor_profile")
        company_research = input_data.get("company_research")
        portfolio_chunks = input_data.get("portfolio_chunks", [])
        # GEMINI CALL 3: Validation
        validation = await gemini.generateContent(f"""
        Do we have sufficient information to personalize?
        Visitor profile: {visitor_profile}
        Company research: {company_research}
        Portfolio chunks: {len(portfolio_chunks)} chunks retrieved
        Return JSON:
        {{
        "sufficient": true/false,
        "confidence": 0.0-1.0,
        "missing_information": [...],
        "next_steps": [...]
        }}
        """)
        return {
            "sufficient": validation.confidence >= 0.7,
            "confidence": validation.confidence,
            "missing": validation.missing_information
        }

```

****AGENT 5: Personalization Agent****

Responsibility: Synthesize all research into complete website config

```

class PersonalizationAgent(Agent):
    """
    Takes all research and generates ENTIRE website configuration.
    Output is stored and sent to frontend.
    Frontend renders based on this config.
    """
    async def run(self, input_data):
        visitor_profile = input_data["visitor_profile"]
        company_research = input_data.get("company_research")
        portfolio_chunks = input_data.get("portfolio_chunks", [])
        # GEMINI CALL 4: Personalization (most complex)
        website_config = await gemini.generateContent(f"""
        Given:
        - Visitor: {visitor_profile}
        - Company: {company_research}
        - Portfolio evidence: {portfolio_chunks}

```

Generate COMPLETE website configuration (JSON):

```
{
  "hero": {
    "intro": "2-3 sentence personalized introduction",
    "subheading": "what matters to this visitor",
    "cta_text": "call to action"
  },
  "featured_projects": [
    {
      "id": "project-1-id",
      "title": "project name",
      "highlight": "why THIS visitor should care",
      "metrics": ["metric 1", "metric 2"],
      "why_relevant": "connection to their company/role"
    },
    {
      "id": "project-2-id",
      "title": "...",
      "highlight": "...",
      "metrics": [...],
      "why_relevant": "..."
    },
    {
      "id": "project-3-id",
      "title": "...",
      "highlight": "...",
      "metrics": [...],
      "why_relevant": "..."
    }
  ],
  "skills_priority": [
    { "skill": "RAG", "priority": 1, "proof": "from portfolio" },
    { "skill": "LangGraph", "priority": 2, "proof": "..." },
    ...
  ],
  "journey_highlights": [
    { "milestone": "Built RAG system", "relevance": "matches their tech" },
    { "milestone": "Production deployment", "relevance": "reliability focus" },
    ...
  ],
  "chat_context": {
    "opener": "opening message for chat",
    "focus_areas": ["topic 1", "topic 2", "topic 3"],
    "avoid": ["topic to de-emphasize"]
  },
  "suggested_queries": [
    "question 1 they might ask",
    "question 2",
    "question 3"
  ]
}

""")
return {
  "visitor_profile": visitor_profile,
  "website_config": website_config
}
```

****PART 3: FRONTEND CHANGES****

****New API Response Structure****

OLD (X Don't use):

```
{
  "hero_intro": "...",
  "featured_projects": [...],
  "chat_opener": "...",
  "suggested_queries": [...]
}
```

NEW (✓ Use this):

```
{
  "personalization_id": "p_sarah@openai.com_2024-06-15-14-32",
  "visitor_profile": {
    "email": "sarah@openai.com",
    "domain": "openai.com",
    "role": "hiring",
    "company": "OpenAI"
  },
  "website_config": {
    "hero": {
      "intro": "Hey Sarah, I see you're building AI infra at OpenAI...",
      "subheading": "Here's what I've shipped that's relevant...",
      "cta_text": "Explore my work"
    },
    "featured_projects": [
      {
        "id": "project-1",
        "title": "Smart Portfolio with RAG",
        "highlight": "Shows end-to-end RAG system",
        "metrics": ["82% precision", "200ms latency"],
        "why_relevant": "Directly relevant to OpenAI's RAG infrastructure work"
      },
      ...
    ],
    "skills_priority": [...],
    "journey_highlights": [...],
    "chat_context": {...},
    "suggested_queries": [...]
  }
}
```

****New Frontend State****

OLD (X Don't use):

```
const [personalization, setPersonalization] = useState(null);
```

NEW (✓ Use this):

```

const [visitorProfile, setVisitorProfile] = useState(null);
const [websiteConfig, setWebsiteConfig] = useState(null);
const [personalizationId, setPersonalizationId] = useState(null);
// Load from API
const response = await fetch("/api/personalize", {
  method: "POST",
  body: JSON.stringify({email, role})
});
const data = await response.json();
setVisitorProfile(data.visitor_profile);
setWebsiteConfig(data.website_config);
setPersonalizationId(data.personalization_id);
// Store in localStorage
localStorage.setItem("personalization", JSON.stringify({
  personalizationId: data.personalization_id,
  visitorProfile: data.visitor_profile,
  websiteConfig: data.website_config,
  timestamp: Date.now()
})));

```

****Chat Widget Change****

OLD (X Don't use):

```

const sendMessage = async (message) => {
  const response = await fetch("/api/chat", {
    method: "POST",
    body: JSON.stringify({
      message,
      visitor_profile: visitorProfile
    })
  });
};

```

NEW (✓ Use this):

```

const sendMessage = async (message) => {
  const response = await fetch("/api/chat", {
    method: "POST",
    body: JSON.stringify({
      message,
      personalization_id: personalizationId, // NEW: Track which version
      visitor_profile: visitorProfile,
      website_config: websiteConfig
    })
  });
};

```

Why: Chatbot knows which version of the website was generated. Can refer back to the specific personalization context.

****PART 4: DATABASE SCHEMA****

****Firestore****

Collection: `personalizations`

```
personalization_id: "p_sarah@openai.com_2024-06-15-14-32"
email: "sarah@openai.com"
role: "hiring"
company: "OpenAI"
visitor_profile: {...}
website_config: {...}
created_at: timestamp
expires_at: timestamp (24h)
ttl: 86400
```

Collection: `sessions`

```
session_id: "sess_xyz"
personalization_id: "p_sarah..."
visitor_journey: [
  {page: "/", time: 0:30},
  {page: "/projects/1", time: 2:15},
  {page: "/explore", time: 4:00}
]
chat_messages: [...]
created_at: timestamp
```

Collection: `analytics`

```
date: "2024-06-15"
total_visitors: 23
companies: {openai.com: 12, google.com: 6, ...}
avg_time_on_site: 240
engagement: {...}
```

****Qdrant (Vector DB)****

Collection: `portfolio`

```
Points:
id: 1
vector: [0.1, 0.2, 0.3, ...] (384-dim embedding)
payload: {
  text: "chunk of portfolio text",
  doc_type: "project" | "blog" | "resume",
  doc_id: "project-1-casestudy",
  keywords: ["RAG", "full-stack"]
}
```

****PART 5: GEMINI LLM CALLS****

Call	Purpose	Input	Output	Cost	When
1	Research Planning	Email + domain	What info needed	\$0.00075	Always
2	Segmentation	Scraped HTML	Structured JSON	\$0.0015	If scraping done
3	Validation	Research results	Confidence score	\$0.00075	Always
4	Personalization	All context	Website config	\$0.003	Always (most expensive)
5+	Chatbot	Message + context	Response	\$0.00075	Per chat message

Total per personalization: ~\$0.006-0.008 (vs \$0.02+ with Claude)

Why Gemini:

- Flash model = fast + cheap
- Pro model = better quality (used for personalization only)
- 10x cheaper than Claude
- Still high quality for structured outputs

****PART 6: NEW TECH STACK****

```
FRONTEND
├─ Render: Next.js 14 + React
├─ State: Zustand (visitorProfile, websiteConfig, personalizationId)
├─ Styling: Tailwind CSS
└─ Animation: Framer Motion
BACKEND
├─ Runtime: FastAPI on Cloud Run
├─ Framework: LangGraph (multi-agent orchestration)
├─ Agent Language: Python
LLM
├─ Gemini 2.5 Flash (planning, segmentation, validation, chat)
└─ Gemini 2.5 Pro (personalization - best quality)
SCRAPING
└─ Playwright (Chromium headless browser)
DATABASES
├─ Firestore (operational: sessions, analytics, personalization cache)
└─ Qdrant (vector DB: portfolio chunks)
CACHING
├─ Firestore (24h personalization cache)
└─ In-Memory (request-level caching)
DEPLOYMENT
├─ Frontend: Vercel
├─ Backend: Google Cloud Run (FastAPI)
├─ Databases: Firestore + Qdrant
└─ Orchestration: LangGraph
```

****PART 7: WHAT YOU DON'T BUILD (YET)****

****Evaluation Layer (Add Later)****

Future (not MVP):

```
RAGAS Evaluation:
├─ Faithfulness (is response accurate?)
├─ Relevance (are retrieved chunks relevant?)
├─ Context recall (did we get all relevant info?)
└─ Context precision (any irrelevant chunks?)
LLM-as-a-Judge:
├─ Personalization quality (how good is the customization?)
└─ Chat quality (is response helpful?)
```

Not needed for Days 1-6. Can add in Days 22-35 (Project 2).

****PART 8: IMPLEMENTATION TIMELINE (Days 1-6)****

Day	Component	Details
1	Backend scaffolding + Agents 1-5 skeleton	LangGraph setup, agent outlines
2	Company Research Agent	Playwright scraping (website, careers, blog)
3	Portfolio RAG Agent	Chunk docs, embed, store in Qdrant
4	Validation + Personalization Agents	Gemini calls 1-4 working
5	Frontend changes	New state structure, new response parsing
6	Integration + Polish + Deploy	Everything working end-to-end, Vercel deploy

****SUCCESS CHECKLIST (Day 6)****

- ☐ Supervisor Agent orchestrates all agents
- ☐ Company Research scrapes website, careers, blog (no LinkedIn)
- ☐ Portfolio RAG returns relevant chunks from Qdrant
- ☐ Validation Agent scores confidence
- ☐ Personalization Agent generates full website config
- ☐ Firestore stores 24h cache
- ☐ Frontend receives and renders complete website_config
- ☐ Chat widget sends personalization_id
- ☐ localStorage stores entire personalization
- ☐ Gemini calls cost <\$0.01 per personalization
- ☐ Live on Vercel with admin dashboard

The new architecture is simpler, cheaper, and more robust than the original PDF.

Ready to code Days 1-6?